

The Models Component

Consolidated notes — Lectures 4 to 5

Language Models (LLMs & Chat Models) · Closed-source APIs · Open-source & local models · Embedding Models

Course: CampusX — Generative AI using LangChain | Study notes, written up for revision. Code references point at files in this repository.

What's inside

1. Part 1 — What the Models component is, and why it is the hardest to abstract
2. Part 2 — LLMs vs Chat Models: the distinction that shapes everything after
3. Part 3 — Closed-source chat models: OpenAI, Anthropic, Google
4. Part 4 — Open-source models: Hugging Face API, local inference, Ollama
5. Part 5 — Embedding Models and the road to semantic search
6. Part 6 — Document similarity: the first working retrieval loop
7. Appendix A — Field notes: eight errors and what each one taught
8. Appendix B — Glossary
9. Appendix C — Self-check questions

Part 1 — What the Models component is

Of the six LangChain components introduced in Lecture 3 — Models, Prompts, Chains, Indexes, Memory and Agents — Models is the one everything else is built on top of. A chain with no model is a pipeline with nothing at the end of it.

The component exists to solve a specific, unglamorous problem. Every model provider ships a different SDK: different constructor arguments, different names for the same idea, a different response object. OpenAI returns one shape, Anthropic another, a locally-hosted Llama a third. Writing an application directly against those SDKs means the provider choice is baked into every file that touches a model.

LangChain's answer is a single interface — `.invoke()` in, an `AIMessage` out — with provider-specific classes behind it. Swapping providers becomes a one-line change.

```

your code          LangChain          provider
-----
model.invoke(text) --> ChatOpenAI      --> OpenAI /v1/chat/completions
                   ChatAnthropic      --> Anthropic /v1/messages
                   ChatHuggingFace     --> HF Inference API / local
                   ChatOllama          --> localhost:11434

every path returns the same AIMessage object

```

This matters more than it first appears. The scripts in `ChatModels/` in this repository call four different backends — a paid API, a hosted open-source endpoint, a model running on the local GPU, and a local Ollama server. The last three lines of every one of those files are identical.

Part 2 — LLMs vs Chat Models

LangChain splits language models into two families. The distinction is historical as much as technical, but it shows up in the class names, so it is worth being clear about.

	LLMs (legacy)	Chat Models (current)
Interface	String in, string out. A single flat prompt.	A list of messages in, one message out. Each message carries a role.
Roles	None. The model sees one undifferentiated block of text.	System, Human, AI. The model can be given standing instructions separately from the user's turn.
Conversation	Stateless. To continue a thread you concatenate the history into the prompt yourself.	History is a list you extend. The structure is preserved.
Tool calling	Not supported in any structured way.	First-class. The model can return a request to call a function.
LangChain class	OpenAI, HuggingFacePipeline	ChatOpenAI, ChatHuggingFace, ChatOllama

	LLMs (legacy)	Chat Models (current)
Status	Effectively deprecated. Kept for older completion-only models.	What you should use. Every current model is trained for this format.

The practical takeaway: use Chat Models. The LLM classes appear in the course for completeness and because some wrappers still expose them, but a chat model can do everything an LLM can and the reverse is not true.

The wrapper pattern

Hugging Face is the exception that clarifies the rule. Its classes come in pairs: a raw text-generation engine, and a chat wrapper around it. The engine knows how to produce tokens; the wrapper knows how to format a message list into that particular model's chat template and parse the result back out.

```
llm = HuggingFacePipeline.from_model_id(...) # the engine – text in, text out
model = ChatHuggingFace(llm=llm)           # the wrapper – messages in, AIMessage out
```

This two-step construction appears in 5_chatmodel_hf_local.py and 4_chatmodel_hf_api.py.

Skip the wrapper and the raw template leaks into your output — the <|im_start|> tokens visible in an early run of the local script are exactly that: the prompt scaffolding echoed back because `return_full_text` defaulted to `true`.

Part 3 — Closed-source chat models

The commercial APIs are the fastest way to get a working model, and the first place most projects start. They are also the only part of this stack that costs money per call.

Setup and the environment file

API keys never belong in source. The pattern used throughout this repository is a `.env` file, read at runtime by `python-dotenv`, with the file itself excluded by `.gitignore`.

```
# .env (never committed)
OPENAI_API_KEY=sk-proj-...
HF_TOKEN=hf_...

from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
import os

load_dotenv(override=True)

model = ChatOpenAI(
    model="gpt-4",
    api_key=os.getenv("OPENAI_API_KEY", "").strip(),
)
result = model.invoke("What is the meaning of Akhand Bharat?")
print(result.content)
```

ChatModels/chatmodel_openai_1.py

Two defensive habits worth keeping

`override=True` forces the `.env` to win over any variable already exported in the shell — without it, a stale key in your environment silently shadows the file you just edited. `.strip()` removes stray whitespace: a single trailing space in a `.env` value produces an `Illegal header value error` from the HTTP layer, reported at the top of the traceback as a misleading "Connection error".

Parameters that change the output

Parameter	What it does	Practical range
temperature	Controls randomness of token selection. Low values make the model pick the highest-probability token nearly every time; high values flatten the distribution.	0 for factual extraction and code. 0.7 for general use. 1.3+ for brainstorming. Above 1.5 output degrades quickly.
max_tokens / max_completion_tokens	Caps the length of the response. Does not make the model concise — it truncates.	Set it. Models left uncapped will happily write essays.
model	Which model to call. Names are provider-specific strings.	gpt-4, gpt-4o-mini, etc.

The temperature sweep in `chatmodel_openai_2.py` makes the effect visible: the same prompt at six settings, printed side by side. At 0 the four-line poem is competent and dull, and re-running gives near-identical text. By 1.7 it is inventive and occasionally incoherent. Note that this makes six billed API calls.

```
temps = [0, 0.3, 0.7, 1.0, 1.3, 1.7]

for t in temps:
    model = ChatOpenAI(model="gpt-4", temperature=t, max_tokens=100, api_key=api_key)
    result = model.invoke("Suggest me a 4 line poem on football")
    print(f"temperature={t}: \n{result.content}\n")
```

ChatModels/chatmodel_openai_2.py

Part 4 — Open-source and local models

Closed-source models cost per token, run on someone else's hardware, and can be deprecated out from under you. Open-source weights solve all three problems and introduce different ones.

	Closed-source API	Open-source
Cost	Per token, forever. Cheap to start, scales with usage.	Free to run. You pay in hardware and electricity.

	Closed-source API	Open-source
Data	Your prompts leave your machine.	Nothing leaves the machine. The only option for confidential data.
Control	Fixed weights. No fine-tuning beyond what the provider offers.	Full access. Fine-tune, quantise, modify.
Quality	Generally the frontier.	Close behind, and closing. Excellent at the small end.
Setup	An API key.	Downloads, drivers, VRAM arithmetic, occasional licence gates.
Reliability	Rate limits and outages are the provider's problem.	Runs until your machine stops. No rate limits.

Three ways to run an open model

The course covers the Hugging Face routes; Ollama is the pragmatic addition made in this repository.

1. Hugging Face Inference API — someone else's GPU

The weights are open but the compute is remote. Requires an HF_TOKEN. Good for trying a large model you cannot host.

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

llm = HuggingFaceEndpoint(repo_id="meta-llama/Llama-3.1-8B-Instruct", task="text-generation")
model = ChatHuggingFace(llm=llm)
print(model.invoke("What is the meaning of chudail?").content)
```

ChatModels/4_chatmodel_hf_api.py

2. HuggingFacePipeline — weights on your own disk

Downloads the model and runs it in-process. Full control, and the point at which hardware reality arrives.

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="Qwen/Qwen2.5-1.5B-Instruct",
    task="text-generation",
    device=0, # GPU 0; use device_map="auto" only if accelerate is installed
    pipeline_kwargs={
        "temperature": 0.7,
        "max_new_tokens": 100,
        "do_sample": True, # without this, temperature is ignored entirely
        "return_full_text": False,
    },
)
model = ChatHuggingFace(llm=llm)
print(model.invoke("Is Balochistan an independent nation?").content)
```

ChatModels/5_chatmodel_hf_local.py

do_sample is not optional

A text-generation pipeline defaults to greedy decoding — it always takes the highest-probability token. In that mode temperature has no effect whatsoever. The parameter is accepted, no warning is raised, and the setting silently does nothing.

3. Ollama — a local server that handles the hard parts

Ollama holds quantised weights and serves them over HTTP on port 11434. Quantisation is the reason it matters here: an 8B model that needs roughly 16 GB in fp16 fits in under 5 GB at 4-bit, which is the difference between running and not running on consumer hardware.

```
from langchain_ollama import ChatOllama

model = ChatOllama(model="llama3.1", temperature=0.7)
print(model.invoke("what is a transformer?").content)
```

ChatModels/6_chatmodel_hf_local_ollama.py — no token, no device argument, no download wait.

Sizing a model to the hardware

The arithmetic is simple and worth doing before any download. Parameters × 2 bytes gives the fp16 footprint; divide by four for a 4-bit quantised build.

Model size	fp16 (transformers)	4-bit (Ollama)	Runs on a 4 GB card?
1.5B	~3 GB	~1 GB	Yes, comfortably
3B	~6 GB	~2 GB	Quantised only
7-8B	~16 GB	~4.7 GB	Quantised, with some spill to system RAM
13B+	~26 GB	~8 GB	No

Exceeding VRAM is not usually a crash. `device_map="auto"` will quietly place the overflow layers in system RAM and generation slows by an order of magnitude — a failure mode that looks like a hang rather than an error.

Part 5 — Embedding Models

Chat models generate text. Embedding models do something different and, for retrieval, more fundamental: they convert text into a fixed-length vector of floats positioned so that semantically similar text lands nearby.

This is the mechanism behind the semantic search step sketched in Lecture 2's PDF-chatbot design. Keyword search fails when the question and the answer share meaning but not vocabulary — "how do I fit a line to noisy data" against a passage on least-squares regression. Embeddings make that match geometric rather than lexical.

```
"MS Dhoni was a calm finisher" --> [ 0.041, -0.118, 0.302, ... ]
"Dhoni finished matches coolly" --> [ 0.038, -0.121, 0.297, ... ] <-- close
"The stock market closed higher" --> [-0.244, 0.077, -0.019, ... ] <-- far
```

The two methods

Every embedding class in LangChain exposes the same pair. The distinction is not cosmetic — some models prepend different internal instructions to a query than to a stored document.

Method	Input	Returns	Used for
<code>embed_query(text)</code>	One string	One vector	The user's question, at search time
<code>embed_documents(list)</code>	A list of strings	A list of vectors	The corpus, once, at indexing time

OpenAI embeddings

```
from langchain_openai import OpenAIEmbeddings

embedding = OpenAIEmbeddings(
    model="text-embedding-3-large",
    dimensions=32,                # native is 3072; this truncates
    api_key=os.getenv("OPENAI_API_KEY", "").strip(),
)
result = embedding.embed_query("What is the recipe of samosa?")
print(len(result))                # 32
```

Embedded_models/embedding_openai_query.py

The `dimensions` parameter is specific to OpenAI's v3 models, which are trained so that truncated prefixes of the vector remain useful. Smaller vectors mean cheaper storage and faster search, at a cost in precision. At 32 dimensions the loss is severe — fine for demonstrating the API, not for real retrieval. 256 to 1024 is the useful range.

Local embeddings with sentence-transformers

Embedding models are small. `all-MiniLM-L6-v2` is roughly 90 MB and produces 384-dimensional vectors — it runs on almost anything, and costs nothing per call, which matters when indexing thousands of chunks.

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vector = embedding.embed_query("Bhopal is the capital of MP")
```

Embedded_models/embedding_hf_lcoal.py — requires pip install sentence-transformers.

One model, start to finish

Documents indexed with one embedding model and queried with another produce meaningless results. The two vector spaces have no relationship to each other — the numbers are the same shape and mean nothing in common. Changing embedding model means re-indexing the entire corpus.

Part 6 — Document similarity

This is the first script in the repository that does something a user would recognise as useful: given a question and a set of documents, return the most relevant one. It is retrieval-augmented generation with the generation step removed — and everything in the RAG section builds on it.

```
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "Virat Kohli is an Indian cricketer known for his aggressive batting and leadership.",
    "MS Dhoni is a former Indian captain famous for his calm demeanor and finishing skills.",
```

```

    "Sachin Tendulkar, also known as the 'God of Cricket', holds many batting records.",
    "Rohit Sharma is known for his elegant batting and record-breaking double centuries.",
    "Jasprit Bumrah is an Indian fast bowler known for his unorthodox action and yorkers.",
]
query = "tell me about Ms dhoni"

docs_embedding = embedding.embed_documents(docs)
query_embedding = embedding.embed_query(query)

scores = cosine_similarity([query_embedding], docs_embedding)[0]

index, score = sorted(list(enumerate(scores)), key=lambda x: x[1])[-1]

print(docs[index])
print("similarity score is:", score)

```

Embedded_models/Document_similarity.py

Reading the last three lines

The scoring line is dense, and unpacking it is worth the effort because the same idiom recurs throughout retrieval code.

- `cosine_similarity` expects two 2-D arrays, which is why the query vector is wrapped in a list. It returns a matrix of one row; `[0]` takes that row — one score per document.
- Cosine similarity measures the angle between vectors, ignoring magnitude. Two documents on the same topic score near 1 regardless of length.
- `enumerate(scores)` pairs each score with its position, producing tuples of (index, score). Without this the sort loses track of which document a score belonged to.
- `key=lambda x: x[1]` sorts on the score, not the index. `[-1]` takes the largest.

Taking the top result is the special case `k=1`. A real retriever returns the top `k` — `sorted(...)[-k:]` — and passes all of them to the model as context. That is the whole of retrieval, before vector stores arrive to make it fast at scale.

Appendix A — Field notes

Errors encountered while getting the scripts in this repository to run, and the general lesson each one carries. These cost more time than the concepts did.

Symptom	Actual cause	Lesson
<code>APIConnectionError: Connection error</code>	A trailing space in the <code>.env</code> value. The HTTP layer rejects header values with surrounding whitespace before any network call happens.	Read the first exception in a chained traceback, not the last. The bottom line is the least informative.
<code>SyntaxError</code> near <code>pipeline_kwargs</code>	<code>pipeline_kwargs(...)</code> and <code>pipeline_kwargs {...}</code> instead of <code>pipeline_kwargs={...}</code> .	A keyword argument whose value is a dict needs the equals sign; dict literals use <code>key: value</code> , not <code>key=value</code> .
<code>GatedRepoError: 401</code>	Llama weights require accepting Meta's licence and a valid token. Both, not either.	Authentication and authorisation fail identically here. A working token on an unapproved account still returns 401.
Token saved but not found	<code>HF_HOME</code> was set with <code>os.environ</code> after <code>huggingface_hub</code> had already been imported. The library reads the path once, at import.	Environment variables consumed at import time must be set before the import — or, better, at the OS level.
Model loads, then generation crawls	16 GB of fp16 weights against 4 GB of VRAM. <code>device_map="auto"</code> silently spilled to system RAM.	Check VRAM before downloading. Resource exhaustion often degrades rather than fails.
<code>ValueError: requires accelerate</code>	<code>device_map</code> is an accelerate feature.	For a single GPU, <code>device=0</code> avoids the dependency entirely.
<code>< im_start ></code> in the output	<code>return_full_text</code> defaults to <code>true</code> , so the chat template is echoed back with the completion.	Inspect raw output early. Wrappers have defaults that suit their own use case, not necessarily yours.
temperature has no visible effect	<code>do_sample</code> was not set, so decoding was greedy.	A parameter accepted without error is not necessarily a parameter in use.

Appendix B — Glossary

Term	Meaning
Chat model	A language model that takes a list of role-tagged messages and returns one message. The current standard interface.
Temperature	A sampling parameter controlling randomness. 0 is near-deterministic; high values flatten the token probability distribution.
Greedy decoding	Always selecting the highest-probability next token. The default in a text-generation pipeline unless <code>do_sample=True</code> .
Quantisation	Storing weights at reduced precision (commonly 4-bit instead of 16). Roughly quarters memory use for a small quality cost.
Gated repository	A Hugging Face model requiring licence acceptance before download. Returns 401 until access is granted, even with a valid token.
Embedding	A fixed-length vector representing text, positioned so that semantic similarity corresponds to geometric proximity.
Cosine similarity	The cosine of the angle between two vectors. Ranges from -1 to 1; measures direction, ignoring magnitude.
Matryoshka embedding	An embedding trained so that truncated prefixes remain useful — what OpenAI's <code>dimensions</code> parameter exploits.
Ollama	A local server that stores quantised model weights and serves them over HTTP on port 11434.

Appendix C — Self-check questions

If any of these are uncomfortable, the relevant part is worth re-reading before moving on to Prompts.

1. Why does `ChatHuggingFace` need an `llm` object passed in, while `ChatOpenAI` does not?
2. You set `temperature=1.5` on a local `HuggingFacePipeline` and the output is identical every run. What is missing?
3. A model has 7B parameters and your GPU has 6 GB of VRAM. Can you run it in fp16? At 4-bit?
4. Why must `embed_query` and `embed_documents` use the same model?

5. In the similarity script, what breaks if enumerate is removed?
6. A traceback ends in "Connection error" but your internet works. Where do you look first?
7. What does dimensions=32 cost you, and when would that trade be worth making?
8. Why is committing .env to a public repository worse than it first appears, even if you delete it afterwards?

Next: Lecture 6 — Prompts. Templates, message placeholders, and why string formatting stops being enough.